EXP.NO: 10	Page Replacement Algorithm
DATE: 17/04/2026	

AIM:

To implement and compare CPU page replacement algorithms (FIFO and LRU) to determine their efficiency by calculating the total number of Page Faults and Page Hits for a given sequence of memory requests.

ALGORITHM:

1. **Input:** Accept the total number of page requests, the sequence of pages (reference string), and the number of available memory frames.
2. **Memory Initialization:** Initialize the memory frames to an empty state (e.g., `-1`).
3. **Traverse Reference String:** For each page requested:
4. * **Search:** Check if the page is already present in the frames.
5. * **Hit:** If found, increment the **Page Hit** count. (In LRU, also update the "last used" time for that page).
6. * **Fault:** If not found, increment the **Page Fault** count and replace a page:
7. * **FIFO:** Replace the page at the current pointer position and move the pointer forward (circularly).
8. * **LRU:** Replace the page that has the oldest timestamp (the one not used for the longest time).
9. **Output:** Display the final counts for Page Faults and Page Hits.

CODE:

FIFO

```
#include <stdio.h>
int main() {
   int n,f,pf=0,ph=0,ptr=0;
   scanf("%d",&n);
   int p[n]; for(int i=0;i<n;i++) scanf("%d",&p[i]);
   scanf("%d",&f);
   int fr[f]; for(int i=0;i<f;i++) fr[i]=-1;

   for(int i=0;i<n;i++){
      int found=0;
      for(int j=0;j<f;j++)
         if(fr[j]==p[i]) { found=1; ph++; break; }

      if(!found){
```

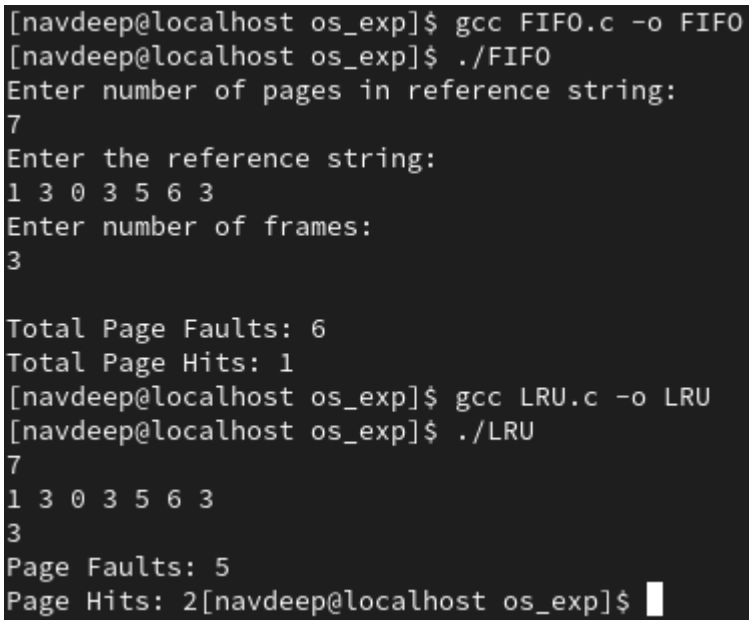
```
            fr[ptr]=p[i];
            ptr=(ptr+1)%f;
            pf++;
        }
    }
    printf("Page Faults: %d\nPage Hits: %d",pf,ph);
}
```

LRU

```
#include <stdio.h>
int main() {
    int n,f,pf=0,ph=0,t=0,c=0;
    scanf("%d",&n);
    int p[n]; for(int i=0;i<n;i++) scanf("%d",&p[i]);
    scanf("%d",&f);

    int fr[f],lu[f];
    for(int i=0;i<f;i++) fr[i]=lu[i]=-1;

    for(int i=0;i<n;i++){
        int idx=-1;
        for(int j=0;j<f;j++)
            if(fr[j]==p[i]) { idx=j; break; }

        if(idx!=-1){
            ph++; lu[idx]=t++;
        } else {
            pf++;
            if(c<f){
                fr[c]=p[i]; lu[c++]=t++;
            } else {
                int lru=0;
                for(int j=1;j<f;j++)
                    if(lu[j]<lu[lru]) lru=j;
                fr[lru]=p[i]; lu[lru]=t++;
            }
        }
    }
```

```
    printf("Page Faults: %d\nPage Hits: %d",pf,ph);
}
```

OUTPUT:



RESULT:

The program successfully simulates process synchronization. The producer is restricted from adding items when the buffer is full (e=0), and the consumer is restricted from removing items when the buffer is empty (f=0), maintaining data integrity within the shared resource.